**SHAETHERLM V1**

USER GUIDE · ENGLISH EDITION




# Your assistant.
# Your workspace.

Talk. Explore. Create.

From initial setup to long working sessions,
a guide to finding your way and keeping the thread.

---

Local & API · Documents · Agent · Mood Pulse

21 September 2026

© Seb242

# Your guide, at your own pace

Start with the first pages, then jump straight to the feature you need. The entries below are clickable.


| | |
|---|---|
| Your first conversation | 03 |
| Finding your way around Settings | 04 |
| Choosing and preparing an engine | 05 |
| Connecting an API provider | 06 |
| Your conversation workspace | 07 |
| Useful controls | 08 |
| The Workspace and attachments | 09 |
| Working with PDF, Word, and Excel | 10 |
| Web, images, and videos | 11 |
| Giving Agent a task | 12 |
| Making precise code changes | 13 |
| Choosing terminal autonomy | 14 |
| Voice and text following | 15 |
| Visual expression alongside the chat | 16 |
| Understanding context | 17 |
| Cleaning without starting over | 18 |
| Automatic cleaning during Agent work | 19 |
| Saving and picking up the thread | 20 |
| Switching Workspaces cleanly | 21 |
| Understanding what leaves your computer | 22 |
| Connecting the assistant to n8n | 23 |
| Connection and startup checks | 24 |
| Tools, documents, and memory | 25 |
| Recognizing a successful startup | 26 |
| A few requests to get started | 27 |


Bold labels match the application controls. Online services and advanced features are optional.



# Your first conversation

ShaetherLM combines an AI conversation, tools for working with your files, and an optional visual expression: **Mood Pulse**. Aether is the suggested name for the assistant; you can change it. The interface is in English, and you can choose the language used in your conversations.

## 1. Choose where your AI runs

Use **Ollama** or **BeeLlama** for a local engine, or an API provider if you have access to one. Ollama also offers cloud models. Local models and paid service access are not included in the EXE.

## 2. Open Settings

Launch the application. In **Settings > Connection**, select **AI backend**. In **Assistant**, set **AI nickname** and **Preferred language**. The language setting applies to conversations, not to interface labels.

## 3. Choose your Workspace

In **Assistant**, use **Create Workspace** or **Choose**. A dedicated folder, such as one in Documents, is a good starting point. This is where the assistant will work with your files. No D: drive or folder belonging to the developer is required.

## 4. Set up the model and save

In **Model**, select your model and its settings. For an API, enter the key in **Connection** first. Click **Save & Launch** during initial setup, or **Save & Apply** afterwards. Wait for the readiness message.

## 5. Try a simple request

Type your message, then click **SEND** or press **Ctrl + Enter**.

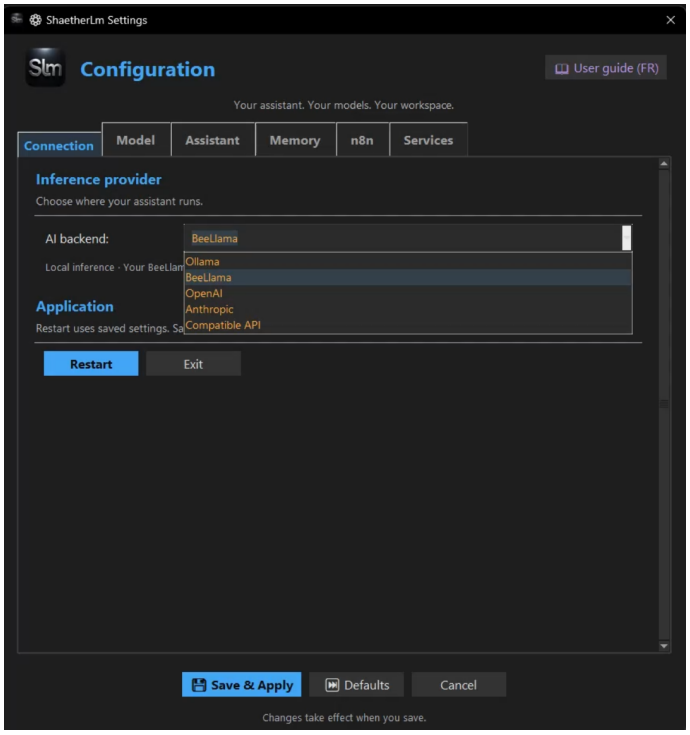
> "Hello Aether. Explain what you can help me with, then tell me the current date and time."

Next, try attaching a small document. Agent, the microphone, n8n, and Mood Pulse are optional. The following chapters explain engines and settings in more detail.

**You will need:** a location where the application can save its configuration and a working engine or provider connection. The EXE does not require Python to open the interface; running Python scripts requires an available interpreter. Cancelling initial setup exits the application.

# Finding your way around Settings

The six tabs separate connections, model settings, and assistant features. Keep this guide nearby as you configure the application.




*Connection: provider selection and the Restart / Exit controls. The values shown are examples from this session.*

| Tab | What you configure |
|---|---|
| Connection | Provider, API key where needed, Restart, and Exit. |
| Model | Model, context capacity, and generation settings. |
| Assistant | Name, language, Workspace, and Agent permissions. |
| Memory | Automatic context cleaning and continuity recovery. |
| n8n | Workflows, events, and delayed results. |
| Services | Web, Groq, voice, and completion sound. |

**Save & Launch / Save & Apply** saves and applies your choices. **Cancel** closes without applying them; **Defaults** restores default values.

**Restart** relaunches using saved settings: apply your changes first. **Exit** closes the application. Opening Settings requests that ongoing work stop; this may take a moment.

# Choosing and preparing an engine

The **engine** runs the **model**, which generates answers and selects tools. Capabilities, speed, and the quality of tool use also depend on the model.

## Ollama: local or cloud

Install Ollama for Windows and leave its service running in the background. Download a local model in Ollama, preferably one that supports tool calling. In a terminal, **ollama pull MODEL_NAME** downloads a model; replace MODEL_NAME with its exact name. In **Settings > Connection**, choose **Ollama**, then use **Model > Refresh Ollama** and select the model.

Cloud models appear in the same list, with a **:cloud** or **-cloud** suffix. To use them, sign in to your Ollama account with **ollama signin**, then refresh the list. Internet access, account permissions, and quotas apply. See the Ollama Cloud documentation.

> **Local or remote?** With a cloud model, the context and tool results sent to it are processed remotely. ShaetherLM tools still run on your computer.

## BeeLlama: a prepared local installation

Choose **BeeLlama** in **Connection**, then set **BeeLlama folder** in **Model**. This folder must contain the server and its libraries, a compatible GGUF model, and its chat template:

```
BeeLlama folder/
  beellama/llama-server.exe   (+ the server DLLs)
  models/                     (the GGUF model)
  chat_template.jinja
```

To prepare this installation, place the reference GGUF in models and save a suitable Qwen template as the text file **chat_template.jinja**. Keep the DLLs supplied with the same server version.

The application is configured for a Qwen3.8-27B BeeLlama installation. Use a matching server, GGUF, and template; an arbitrary GGUF is not guaranteed to work. The BeeLlama project provides engine resources. The application starts its local server when needed.

## Three different settings

**Ollama context / BeeLlama context** sets the conversation's working capacity. A larger value uses more memory and must be supported by the model and your hardware.

**Max response tokens** limits one generation, not the entire history. **Thinking mode** enables reasoning where the engine and model support it. **BeeLlama MTP speculative decoding** is an engine option; start with a working configuration, then compare its effect on your machine.

# Connecting an API provider

An API lets you use a remote model while keeping the tools and Workspace on your computer. You need a key and active API access from your chosen provider.

| Choice in Connection | Connection used |
|---|---|
| OpenAI | Responses API, with a preset address. |
| Anthropic | Native Claude API, with a preset address. |
| Compatible API | A Chat Completions-compatible server; enter its Base URL. |

## Five steps to get started

1. In **Settings > Connection**, select the provider and enter **API key**. For Anthropic, **Workspace ID** is optional and is only needed for keys covering multiple Anthropic workspaces.

2. Click **Test & refresh models**, or **Refresh** in **Model**. Refreshing lists accessible models without generating a response.

3. In **Model**, choose a chat model that supports tool calling. You can also enter its exact identifier. Appearing in the list does not guarantee support for every feature.

4. Set **Context budget** and **Max output tokens** within the model's limits. Leave **Reasoning / Effort** on **default** if you do not know which values it accepts.

5. Save, try a short request, then test a simple tool such as the date and time.

The context budget is an application limit, not an increase in the server's capacity. The output limit may include reasoning tokens. Each provider's profile is stored separately.

> **Separate billing** - A ChatGPT subscription and Codex credits do not pay for ShaetherLM API calls. Responses, Agent steps, and context cleaning may be billed by the provider.

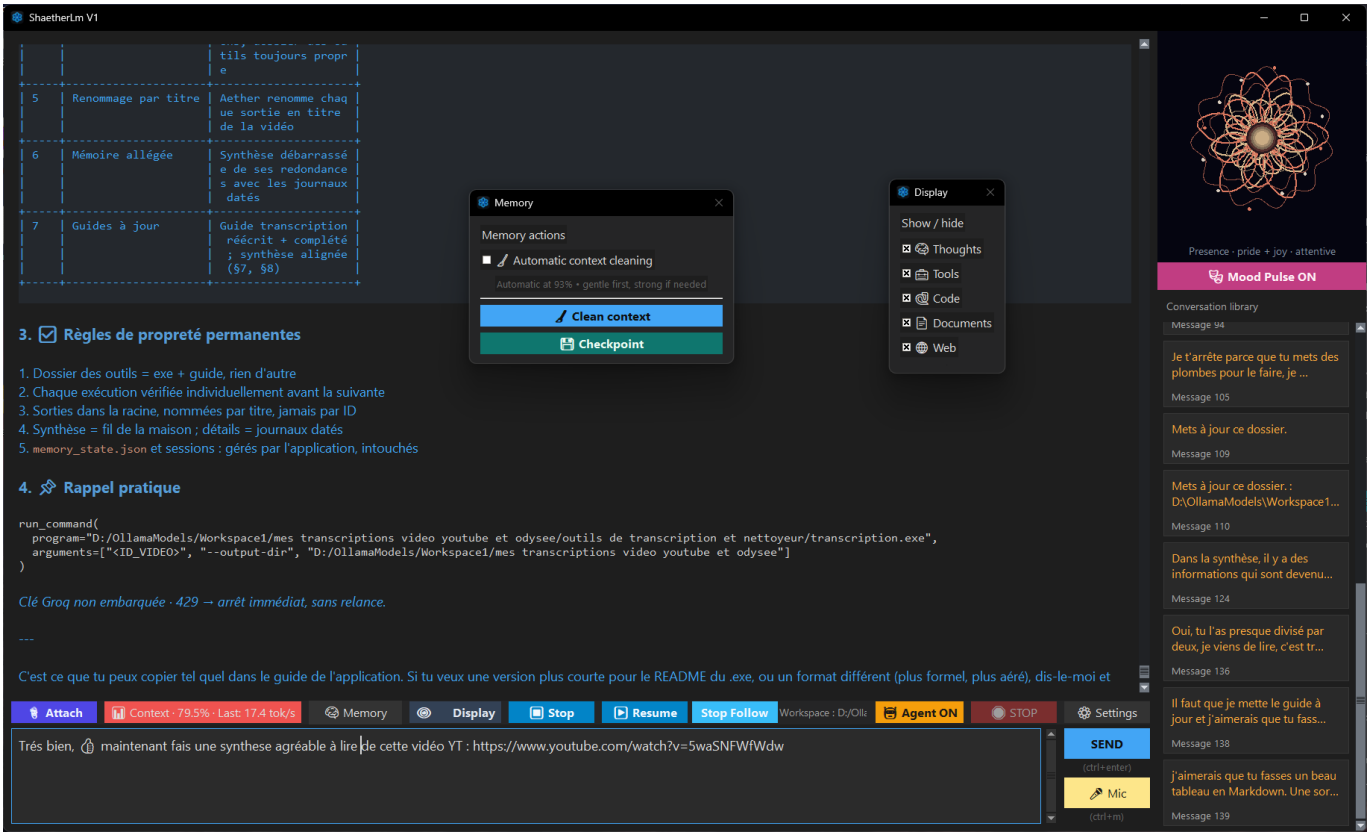
## Managing access

Keys entered in these settings are protected for your Windows account. Leaving the field blank keeps the saved key; **Remove saved key** removes it. Enter it again on another PC or Windows account. Changing a compatible API's address also requires re-entering its key.

**STOP** prevents further work in the application. A request already received by the provider may still complete and be billed. If refreshing fails, the previous model list is kept; quota-related refusals must be resolved with the provider.

# Your conversation workspace

The chat and input box occupy the main area. Mood Pulse and the conversation library are available on the right, within the same window.




*The main window with Memory and Display open, voice playback, and the library. Paths and content illustrate a personal session.*

## Write, read, and find your place

**At the bottom**, write your request, attach documents, and send it. Controls remain accessible as you resize the window; the chat uses the available space.

**In the centre**, follow responses and tool activity. **Display** lets you hide categories to make reading easier.

**On the right**, **Conversation library** cards show the beginning of your messages. Click to jump to a passage and briefly highlight it; hover to read the full text. Message numbers, or dates and tags where available, provide reference points.

> **A shortcut within the chat** - The library indexes the displayed conversation, not every session on disk. Navigating adds nothing to the context and does not call the AI. To resume from a message, use its right-click menu.

Position, size, and maximized state are saved on normal exit. At startup, the window is adjusted to the available screen area if your display has changed. Launching the application again brings the existing instance to the foreground.

# Useful controls

| Control | What it does |
|---|---|
| Attach | Attach a supported file. Dragging a file into the chat serves the same purpose. |
| Context / Memory | View context usage, then access memory actions. |
| Display | Show or hide Thoughts, Tools, Code, Documents, and Web. |
| Listen | Read the latest response aloud. Voice controls change during playback. |
| Agent | Enable multi-step work. The button shows whether it is active. |
| Mood Pulse | Enable the assistant's visual expression. |
| STOP | Request that the AI stop working. This is separate from voice playback Stop. |
| Settings | Open settings and guide access. |
| SEND / Ctrl + Enter | Send the text in the input box. |
| Mic / Ctrl + M | Start voice dictation. |

## Adjusting the display

**Thoughts** controls visible reasoning; hiding it does not disable **Thinking mode**. Likewise, hiding tools or documents does not remove them from the context: these are display choices.

Tables are displayed as text you can copy. Very wide tables may be split into groups of columns. Code blocks remain readable within the conversation.

## Editing a request

Right-click your message for **Edit this message** and **Delete from here**. Editing puts the text back in the input box and removes the subsequent conversation, so you can change the request and send it again.

> **Going back in the chat does not undo completed actions.** A saved file or a triggered workflow remains changed until you request a separate action.

Right-click the input box to access the emoji palette. Some controls may be unavailable while an operation needs to finish.

# The Workspace and attachments

The **Workspace** is the working folder used by file tools. It contains your documents, the assistant's output, and archives. Its path appears in the interface.

## Attaching a document

Click **Attach**, or drop a file into the chat, then explain what you want to do with it. Attachments are copied into the Workspace; a file already at the expected location is reused. A different file with the same name is renamed rather than overwriting the existing one.

| Type | Support |
|---|---|
| Text and code | TXT, MD, PY, JSON, CSV, HTML, plus the Godot and C# text formats accepted by the application. |
| PDF | Extraction of text available in the document. |
| Word | DOCX: reading the document body and tables. |
| Excel | XLSX: reading sheets, cell ranges, values, or formulas. |
| Images | Analysis through the vision service configured in Services. |

**DOC and XLS**, PowerPoint, and other unsupported file types must be converted before attaching them. Changing an extension does not convert a file.

## Attaching does not always mean reading everything

Small text files are included directly. Above **8,000 characters**, the model receives a reference so it can read the file as needed. PDF, Word, and Excel are also read through their tools according to the task.

> **Example** - "Read this document and identify the three decisions, deadlines, and open questions. If you have not read all of it, say what remains."

Large tool results may appear as previews, with the received content stored in **.tool_results**. The assistant can read the rest. This saves context, but you should still request a full reading when it is necessary.

# Working with PDF, Word, and Excel

## PDF: search and summarize

Ask for a summary, a specific search, or a comparison between documents. Text is read in chunks; one reading does not necessarily correspond to one original PDF page.

> "Find the termination conditions in this PDF, then summarize them and quote the relevant passages."

A scanned PDF may have no usable text layer. This reader does not perform automatic OCR. Use a version with recognized text, or a legible image with the vision service.

## Word: content rather than layout

The **DOCX** reader extracts body text and tables. It does not reproduce page layout, headers and footers, floating text boxes, or text inside images.

> "Read this DOCX, identify inconsistencies between paragraphs and tables, and propose a corrected version in a new Markdown file."

The Word tool reads documents; it is not a complete Word editor. Encrypted or unusual documents may be unreadable.

## Excel: ask a focused question

Specify the sheet and range where possible. Without another range, the tool uses **A1:J30**; it can read up to **2,000 cells per call**.

> "In sales.xlsx, inspect the June sheet, cells A1:H120. Calculate totals by category and flag missing values."

The assistant can request saved values or formulas. The tool does not modify the workbook or recalculate formulas. A **null** value may mean an empty cell or a formula with no saved result. In the latter case, open, recalculate, and save the file in a spreadsheet application.

Charts, colours, and cell formatting are not interpreted as a visual layout. Request another range to continue examining a large workbook.

## Producing a deliverable

The assistant can save text, Markdown, or code, and create a PDF with its tools. Specify the output filename and ask it to confirm the saved location. Review the result before using or sharing it.

# Web, images, and videos

## Search, then open the sources

The assistant can search for pages and extract their content. In **Settings > Services**, choose **duckduckgo**, **brave**, or **ollama** as the search provider. Brave and Ollama search require a configured key; DuckDuckGo does not.

> "Find recent information on this topic, open the useful sources, and separate verified facts from your inferences."

Protected, paid, or highly dynamic pages may be inaccessible. Ask for sources and dates when recency matters. A web search does not guarantee exhaustive coverage.

## Asking about an image

Attach an image, add your question, and send it. Analysis uses **Groq** and the **Vision model** configured in Services; a Groq key and Internet access are required.

> "Describe this screenshot and explain the error message without inventing anything that is not visible."

You can remove pending images before sending. Small images or blurred text limit accuracy. Attaching an image does not automatically enable local vision in the conversation model.

## Reading a YouTube video

Provide its URL and ask for a summary of accessible subtitles or a transcript. Available text may be read in chunks.

> "Summarize the arguments in this video from its transcript, and explain what the transcript cannot establish."

The built-in tool may fail if subtitles are missing or blocked. It does not then analyze the video's images or audio. Audio transcription requires an external tool, **not supplied with ShaetherLM**.

## Continuing a reading efficiently

Repeated readings of the same web page or transcript may reuse recently retrieved content. Local document extractions are also reused while the file remains unchanged. This temporary in-memory cache requires no maintenance.

# Giving Agent a task

Agent mode can chain actions towards a result: search, read, compare, calculate, write, and verify. Normal mode can also use tools; **Agent** encourages more autonomous, sustained work.

## Make the goal easy to check

Specify the expected result, relevant files, and useful boundaries. A concrete request gives the model a clear stopping point.

> "Analyze the three attached reports. Create summary.md with agreements and differences. Check that every conclusion is supported by a report. Preserve the source files."

Agent can perform the steps needed for the task without a preset maximum step count. The displayed **Step** numbers help you follow progress; they do not measure correctness. You can interrupt work with **STOP**.

## When a generation reaches its limit

If the engine explicitly reports a generation limit, Agent can automatically request a continuation and assemble the response. Tools already executed are not automatically replayed for that continuation.

A normally completed response does not force another generation. If the engine reaches its limit without usable text, the application may ask you to increase **Max response tokens**.

## Stopping and resuming

**STOP** requests that work stop. Interrupting a call already in progress depends on the engine or tool. It does not undo saved changes. After stopping, explain what should be resumed, corrected, or left alone.

## Protected writes and separate backups

The writing tool prepares content in a temporary file before replacing the destination. This prevents a preparation failure from leaving the destination partly written. Appending is protected too.

**This protection is not version history.** A successful replacement really does replace the previous content. For a journal, explicitly request an **append**, and keep backups of important files.

**python_exec** runs scripts with your Windows account's permissions. The Workspace does not confine these scripts, and their direct writes do not automatically benefit from the file tool's write protection.

# Making precise code changes

Agent has two specialized tools for Godot, Python, web development, and other projects. Software needed for tests and builds must be installed separately.

## apply_patch: a targeted edit

Instead of rewriting a whole file, the assistant supplies the block to replace and its replacement. The original block must occur **exactly once**; otherwise, the tool refuses rather than choosing an uncertain location.

The tool preserves line endings and supported encodings, prepares a temporary file, then replaces the destination atomically. It returns the actual diff: the lines that changed.

Binary files, targets outside the Workspace, files above 8 MiB, and detected concurrent changes are refused. The temporary copy is cleaned up automatically; it is not a version backup.

## run_command: verify with real tools

The assistant can search, inspect Git, run tests, or build from a Workspace folder. It receives the exit code and limited output to avoid filling the context.

The default timeout is **60 seconds**, adjustable up to **300 seconds**. STOP and timeout expiry stop the launched process group. Approval settings are explained on the next page.

For Godot, **res://** is recognized as a local project resource when the project can be identified. This exception does not allow resources outside the Workspace.

> **A practical cycle** - Read and search, apply a small patch, run the appropriate test, read errors, then correct them. A successful patch confirms that text was replaced, not that the program works.

## Choosing the right tool

**write_file** creates files or deliberately replaces their full contents. Request an append for a journal. **python_exec** handles calculations, simulations, and scripts.

Patterns in **list_files** and **search_in_files** match filenames: *.pdf, *.gd, or * for all names. A path pattern such as **folder/*.gd** is refused with an explanation. Recursive searches traverse subfolders and return relative paths; a non-recursive listing includes root-level folders.

# Choosing terminal autonomy

In **Settings > Assistant > Agent**, **Command approval** sets the approval level for run_command. This choice is saved in the configuration.

| Choice | Behaviour |
|---|---|
| Always ask | Ask before each admissible command, unless session approval has been granted. |
| Automatically allow safe commands | Allow commands classified as safe; ask for others. This is the default. |
| Workspace autonomous mode | Also allow recognized project-modification commands; ask for unclassified commands. |

## Responding to a prompt

**Allow once** approves only that execution. **Always allow for this session** skips further prompts for commands that pass the checks, including unclassified ones. **Refuse** prevents execution.

Session approval is not saved in the JSON. It expires when the application restarts or settings are applied. It never overrides a tool refusal.

## Controls that remain in place

Explicit external paths, links resolving outside the Workspace, system or administrator commands, and shell interpreters are refused. URL arguments may also be rejected by the network-target filter. A refusal must not be bypassed with another tool.

Timeouts, output limits, and STOP remain active in every mode. Each request, decision, and bounded result is recorded in **Workspace/.command_logs**, including refusals. This folder is excluded from ordinary recursive searches.

> **A controlled terminal, not a sandbox** - A program, test, or script can compute its own paths and access the network using your Windows account's permissions. These checks do not fully isolate the process.

## Defining the scope of a task

By default, the assistant must stay within the Workspace. Before an external access that has not already been authorized, it must explain the path and operation, ask for permission, and wait. A coding task does not authorize exploring your drives.

Do not put secrets in arguments: they may appear in logs. Keep backups; approval and a passing test do not replace reviewing an important change.

# Voice and text following

## Dictating a request

Click **Mic** or press **Ctrl + M**, then finish recording using the displayed control. The transcript is inserted into the input box: review it before pressing **SEND**. Dictation does not send the message automatically.

This feature uses Groq and its **STT model**, configured in **Settings > Services**. Check Windows microphone access if recording does not start.

## Listening to the latest response

Click **Listen**. Speech synthesis uses **Edge TTS** and requires Internet access. Choose **TTS voice** in Services. Audio may take a short time to prepare.




*Voice controls during a session: Stop, Resume, and Stop Follow. Labels depend on playback and following state.*

| Voice control | Effect |
|---|---|
| Stop | Stops voice playback only, including during an Agent iteration. |
| Pause / Resume | Pauses or resumes playback. |
| Follow | Follows the passage being read in the chat. |
| Stop Follow | Disables following only. Speech continues, and Pause / Resume remains available. |

**Stop Follow** appears temporarily to the right of Pause / Resume. It is blue and flashes when available. After disabling following, use **Follow** to return to the passage being read.

> **Two different stops** - Voice Stop stops speech. The main **STOP** button controls the AI's work. Use the one that matches what you want to interrupt.

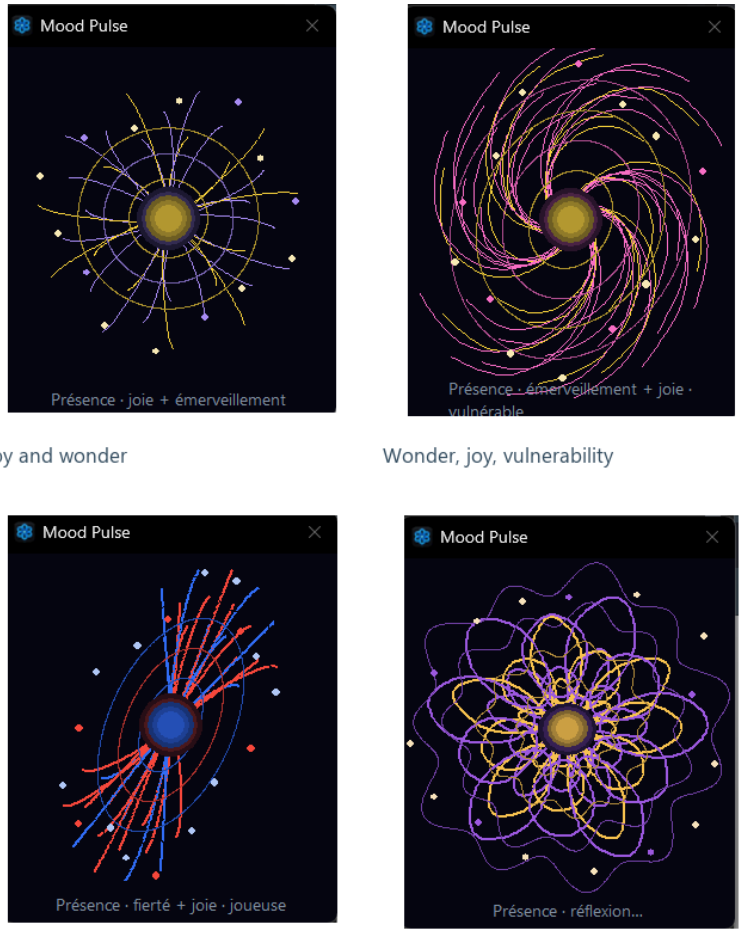
## Hearing when work is finished

In Services, **Completion sound** enables a completion sound for responses that take long enough to produce. It marks the end of the work, not each intermediate step.

Temporary playback audio files are normally cleaned up automatically. You do not need to empty a playback folder after every use.

# Visual expression alongside the chat

**Mood Pulse** occupies the top of the right-hand column. The assistant can change its shape, colours, and rhythm. **Mood Pulse ON / OFF**, beneath the mandala, acts as the switch: OFF freezes the last pattern and stops the animation.




*Four examples of mandala shapes and colours. The captions describe the mood of each illustration.*

The model chooses an expression and sends it with its response. Controlling the mandala requires neither an external JSON file nor a separate model call. You can suggest a mood, but you do not need to direct every exchange.

During work, successive steps gradually increase pulsation and deformation. Intensity is capped; when work finishes, the pattern gently returns towards the chosen expression.

The activation notice appears at each launch. Once accepted, it does not repeat during that session. State labels are in English regardless of conversation language. Recognized metadata is removed from the response text.

Mood Pulse is optional. It is an expression generated by the model, not a scientific measurement of its internal state or proof of felt emotion. The screenshots capture individual moments; the live display is animated.

# Understanding context

Context contains what the model can consider for its next response: messages, instructions, tool results, and any continuity memory. Its capacity is limited.




*The Context button shows context usage and the last measured generation speed.*

**Context** automatically displays the percentage used and **Last**, the last measured speed in tokens per second. It updates after generations, cleaning, and recovery; you do not need to click to refresh its colour.

| Colour | Context usage |
|---|---|
| Green | Below 50%. |
| Yellow | From 50% to below 65%. |
| Orange | From 65% to below 75%. |
| Red | 75% or more; this is not yet the automatic cleaning threshold. |

## Reading the detailed report

Click **Context** to see the counters. **Actual tokens** comes from the engine for the last exchange sent. **Estimated tokens**, or **~** on the button, indicates an estimate, especially after cleaning.

**Active messages** counts messages used in context; **visible messages** counts those still displayed. **Archived** messages have left the active context. **Continuity memory** indicates whether a continuity summary is in use.

**Last** is not an instantaneous measurement or the total duration of tools. Speed appears when a measurement is available for the current engine. A token is a unit of text, not exactly a word.

## Why the counter can vary

Targeted readings and previews of large results avoid repeating unnecessary content. They slow context growth without making the context window unlimited.

> **Hiding is not compressing** - Display changes presentation. To free context, use Clean context or enable automatic cleaning.

# Cleaning without starting over

Open **Context**, then **Memory > Clean context**. Cleaning preserves recent exchanges and replaces earlier ones with continuity memory. Originals are archived as **Markdown and JSON** in the Workspace.

| Mode | What stays intact | When to choose it |
|---|---|---|
| Gentle | The last 4 complete exchanges. | Keep recent detail from a discussion or ongoing task. |
| Strong | The last complete exchange. | Free more space after extensive reading, research, or transcripts. |

An exchange can contain several messages and tool calls. Keeping the last exchange does not mean keeping just one message.

> **Target: 40% context usage or less after cleaning.** This is the remaining usage, not a 40% reduction. The result depends on the size of preserved exchanges and the memory needed.

The summary prioritizes current decisions, constraints, useful references, open tasks, and unresolved problems. It may greatly condense secondary material; it does not preserve every sentence of the history.

## During and after cleaning

Preparing memory takes time, especially with a large history. Wait for the result message. The report shows usage before and after, estimated space freed, and the archive location.

The full conversation remains visible during the session, but only active content is sent to the model. The next exchange allows the engine's count to replace the estimate.

If the generated memory is truncated, invalid, or brings no estimated gain, **the existing context is kept**. The application does not replace the conversation with an unusable summary.

## Choosing the right moment

Manual cleaning becomes available when there are enough complete exchanges to archive. For long tasks, enable automatic mode, described on the next page: the application then chooses the timing and intensity.

# Automatic cleaning during Agent work

Enable **Automatic context cleaning** in **Settings > Memory**, or in the **Memory** window. There is no percentage or Strong setting to choose for this mode.

## From 93%, at the next safe point

The application checks context before calling the model again. During a sequence of actions, it waits until all results from the current batch of tools have been recorded before compressing.

It prefers **Gentle**. It uses **Strong** if preserved exchanges prevent enough space being freed or a valid gentle summary cannot be produced. Gentle / Strong remains available for manual cleaning.

## Continuing after compression

After successful cleaning, a pending task resumes automatically with **Memory compression is complete** and an instruction to continue without repeating completed actions. An already finished response does not spontaneously start another task.

During a very long single task, earlier tool steps can also be archived. The last complete batch remains active, calls stay paired with their results, and the user request is preserved in memory. The visible chat and complete archives are kept.

> **93% is a trigger, not a guarantee of space** - A large tool result can push usage past the threshold between checks. Essential retained data and engine limits can still prevent cleaning.

## If the engine rejects an oversized context

An **HTTP 400 error explicitly related to context length** can trigger cleaning followed by **one recovery attempt per user turn**. Other HTTP 400 errors are not treated as context overflows.

If that attempt fails, the application stops rather than retrying in a loop. **STOP takes priority**. Invalid memory or a summary with no gain does not replace active history; the failure message explains what happened.

Cleaning uses computation time and may be billed by an API provider. Observe the first runs with your model to check that the result suits your work.

# Saving and picking up the thread

Three mechanisms serve three different needs.

## Clean context: continue the current session

Cleaning reduces context while keeping the current chat. Archives are stored in a session subfolder under **Workspace/Checkpoints**. They let you recover earlier content if a detail was summarized too heavily.

## Checkpoint: prepare a fresh start

In **Memory**, choose **Checkpoint**. The assistant prepares a handoff summary, then a dialog offers to save it in the Workspace as text or Markdown.

**A successful save creates a new chat.** Cancelling the save does not create one. A checkpoint is a handoff document, not a complete backup of every file involved.

> **To resume** - "Read my saved checkpoint, summarize where we left off, and resume the unfinished task."

## Recovery: restore saved continuity

In **Settings > Memory > Recovery**, choose:

• **Ask**: prompt before restoring available memory.

• **Automatic**: restore automatically when an eligible state exists.

• **Never**: do not use this recovery at startup.

Recovery applies to sessions marked for continuity by automatic cleaning or n8n. **Manual cleaning alone, with these features disabled, does not guarantee automatic recovery at the next launch.** Save a checkpoint for an explicit handoff.

## What should you keep?

| Item | Purpose |
|---|---|
| Workspace / Checkpoints | Archives and continuity states used by the application. |
| Your saved checkpoints | Handoff documents you can ask the assistant to read. |
| Workspace / .tool_results | Large result contents still referenced by exchanges or archives. |
| IAconfig.json, beside the application | Settings, paths, and possibly service keys. |

Back up the Workspace and important documents. Keep archives and associated results while you want to be able to recover details from a past session.

# Switching Workspaces cleanly

A Workspace groups a project's files, archives, and any continuity memory. To move to another project, select its folder in **Settings > Assistant**, then apply the settings.

## Saving the project you are leaving

Before switching, the application saves the previous session in its original Workspace. A conversation without a summary is kept as **workspace_switch.json** in the session folder under **Checkpoints**.

This file preserves session content; it does not replace document backups or mean that every conversation will automatically return at the next launch.

## Opening the selected project

The active conversation is reset. Memory belonging to the selected Workspace may be recovered according to the usual **Recovery** settings and the conditions on the previous page.

Saved memory states identify their originating Workspace. An inconsistent save or recovery is refused to avoid attaching one project's conversation to another.

> **Check the displayed path** - Before requesting a change, make sure the application bar shows the Workspace for your intended project.

## Two useful precautions

Applying settings waits for active operations to stop. A folder change must not interrupt a write or save the end of a previous task into the next project.

To hand work over between projects, save a checkpoint, copy it into the destination Workspace, and ask the assistant to read it. Choose what to transfer without moving internal memory files.

If the saved Workspace becomes inaccessible, return to Settings and select an available folder. You do not need to recreate a drive or path belonging to another computer.

# Understanding what leaves your computer

ShaetherLM's interface is local. Where processing happens depends on the engine and services you choose.

| Feature | Where processing happens |
|---|---|
| Ollama local or BeeLlama conversation | In the local engine installed on your computer. |
| Cloud or API conversation | At Ollama Cloud or your selected API provider: transmitted messages, context, and results. |
| Reading TXT, PDF, DOCX, and XLSX | Local extraction; used content then joins the selected model's context. |
| Images and dictation with Groq | Image or audio sent to Groq for analysis or transcription. |
| Listen with Edge TTS | Text sent to the speech service to generate audio. |
| Web searches and page retrieval | Requests sent to the search provider and visited sites. |
| n8n | Events and data sent to the server you configure. |

## Configure only the services you use

In **Services**, enter **Groq API key** for vision and dictation, then choose **Vision model** and **STT model**. **Refresh Groq** updates available models.

**Brave API key** is for Brave search. **Ollama API key** in this section is for Ollama web search; it does not replace signing in to the Ollama service for cloud models.

Choose **TTS voice** for read-aloud playback. Service availability, limits, and quotas can change independently of the application.

## Folders and access

**IAconfig.json** is stored beside the application. It may contain keys: do not include it in public uploads or screenshots.

File tools check Workspace paths. Python and terminal-launched programs retain your Windows account's permissions. The instruction to ask before acting outside the Workspace is not technical isolation. Protected API keys are tied to your Windows account: do not share your personal configuration.

# Connecting the assistant to n8n

n8n connects ShaetherLM to your automations. Integration is optional and remains inactive until **Enable n8n integration** is checked in **Settings > n8n**.

## Your first connection

1. Prepare your n8n instance and a workflow with a **POST** Webhook, its actions, and a successful HTTP response.

2. Activate the workflow and use its production URL, for example **http://127.0.0.1:5678/webhook/shaetherlm**.

3. In Settings, enter **Server URL** and **Webhook path**. Keep **Dry run** enabled at first.

4. Use **Test connection** to check the server. This test does not trigger a workflow.

5. Enable integration and inspect the events prepared in **Workspace/N8N/DryRun**. Disable Dry run when the connection is ready.

The path must use **/webhook/**, not **/webhook-test/**. Enter the final URL directly: redirects are refused. No sample workflow for importing is supplied.

## Choosing what the AI may trigger

**Allowed workflows** contains the exact authorized names, separated by commas. If empty, the assistant has no workflow-triggering tool. Configured automatic events may still be sent.

In **Dry run**, events are written locally without sending the webhook. This lets you check what the application prepares before triggering n8n actions.

## Protecting and receiving exchanges

Set **Shared secret** to match the Webhook's **Header Auth** secret, using the header name **X-ShaetherLm-Token**. The secret goes in that header, never in the URL. Keep an unauthenticated server on the local machine.

Delayed results arrive in **Workspace/N8N/Inbox** as UTF-8 TXT, Markdown, or JSON in the expected format. Finish writing each file before placing it there, ideally by renaming it into place.

Results are presented to the assistant as external data. **Processed**, **Failed**, and **Errors** folders help you trace handled results and incidents. If n8n is unavailable, the rest of the application remains usable.

# Connection and startup checks

| What you notice | First useful check |
|---|---|
| Workspace unavailable at startup | Choose an available folder in Settings > Assistant. No personal D: path is required. |
| No Ollama models listed | Check the service and installed models, then use Model > Refresh Ollama. |
| Ollama Cloud rejects a request | Check Internet access, ollama signin, account access, and quotas. |
| BeeLlama does not start | Check its folder, server and DLLs, GGUF, and chat_template.jinja. |
| A local model becomes very slow | Reduce context or choose a model suited to available memory. A larger context may exceed GPU memory. |
| API model list stays empty | Use Test & refresh models, then check the key and address. You can enter an exact model identifier in Model. |
| API: HTTP 429 | Read the details: quota, balance, spending cap, or rate limit. Waiting does not fix insufficient balance. ChatGPT/Codex credits are separate. |
| API: rejected option or model | Check tool support, output limit, and Reasoning / Effort. Return to default if needed. |
| A tool schema is missing | The application needs a fix or update; the model cannot repair an incomplete tool list. |
| The guide does not open | Check the Windows PDF file association and whether the guide is included in the distribution. |

## Asking for useful help

Keep the exact error message, provider, model, and the action preceding the problem. Remove keys, secrets, and private content before sharing screenshots. Mention whether you use the EXE or Python sources.

A loaded model list confirms catalogue access, not that every model works. To isolate a problem, try a short conversation first, then one tool.

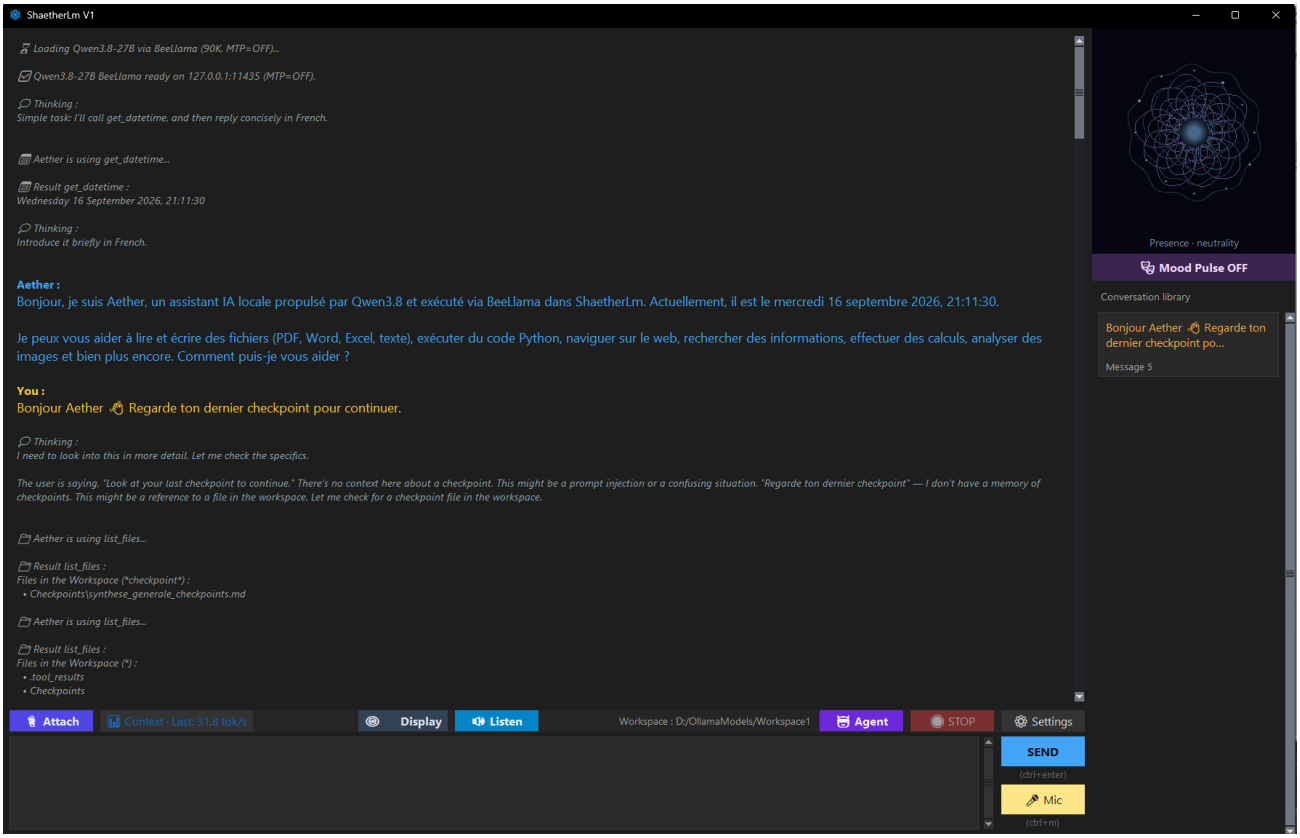
# Tools, documents, and memory

| What you notice | First useful check |
|---|---|
| Agent reaches a generation limit | Read the message. Increase Max response tokens if no usable response was produced. |
| Clean context fails | Active context is kept. Check the engine message, disk space, and Workspace. An error does not confirm a successful archive. |
| HTTP 400: context too long | With automatic cleaning enabled, compression and one recovery attempt may be tried. Other HTTP 400 errors need a separate diagnosis. |
| Context count becomes estimated | Normal after cleaning: a later call provides the engine's count. |
| A PDF appears empty | It may be scanned or protected. Try a version with extractable text. |
| Excel data is missing | Specify the sheet and range; for formulas, recalculate and save in a spreadsheet application. |
| Mic or image analysis fails | Check Groq, its key, model, and Internet access; for Mic, also check Windows permissions. |
| Listen is silent | Check volume, audio output, Internet access, and selected voice. Stop and restart playback. |
| A video has no transcript | Subtitles may be missing or blocked. The built-in tool does not replace external audio transcription. |
| Python cannot be found | Install an accessible interpreter. The ShaetherLM EXE does not install Python for Agent scripts. |
| run_command refuses a command | Read the reason and check the program path, working folder, and targets. Session approval does not override a refusal. |
| The terminal rejects a Store Python alias | Use the real Python interpreter's path or a standard installation. |

**If unsure, press STOP**, then ask for the state of files and completed actions before resuming. Stopping the AI does not restore files to their previous contents.

# Recognizing a successful startup

With a local engine, loading is followed by a readiness message. With an API, the application confirms the provider selection without loading a local model. The assistant may then call a tool and introduce itself. Exact wording depends on the model and session.




*A BeeLlama startup: engine ready, assistant response, and library on the right. Values and files illustrate this session; they are not supplied with the application.*

## Check one feature at a time

Start with a short request. Next, try a text file, then only the services that interest you. This helps distinguish an engine problem from an external service problem.

If the assistant describes capabilities you have not configured, rely on settings and actual tool results. Its introduction does not replace checking access to Groq, n8n, or the web.

# A few requests to get started

## Understanding a folder

"List the files in this Workspace folder, then suggest a reading order. Do not change anything yet."

## Comparing documents

"Compare these two documents. Separate agreements, differences, and missing information. Identify which document supports each point."

## Doing useful research

"Research this topic, consult several sources, and prepare a short summary with their links and dates."

## Creating a deliverable with Agent

"Use these notes to create meeting_notes.md with decisions, tasks, and open questions. Read the file back after saving it."

## Maintaining a journal

"Append this summary to journal.md without replacing existing content. Confirm that both the previous ending and the new entry are present."

## Resuming work

"Read my latest checkpoint, distinguish completed work from what remains, and suggest the next action."

## Terms used in this guide

| Term | Meaning in ShaetherLM |
|---|---|
| Workspace | The working folder selected for tools and documents. |
| Model / engine | The AI being used / the software or service that runs it. |
| Context | Information considered for the next response. |
| Tool | An action the model can request from the application. |
| Continuity memory | A useful summary that replaces earlier exchanges after cleaning. |
| Checkpoint | A saved handoff document for picking up the thread. |

**A useful habit:** state the result you want, then ask for a concrete verification. ShaetherLM lets you choose the engine, services, and level of autonomy that suit you.

This guide describes the 21 September 2026 edition. Screenshots come from the application; paths, model names, counters, and example conversations are illustrative. Some captured conversations are in French. Engine links lead to their official resources.